专升本 **Java** 程序设计模拟卷 **-** 卷 **11** 深度解析

fifine-专升本

2026 年 3 月 2 日

一、单项选择题深度解析（共 20 小题）

1. 下列哪项是 **Java** 语言的特性？

a) 指针运算　　b) 多重继承　　c) 自动内存管理　　d) 预处理器

答案与深度解析 答案：**C.** 自动内存管理

【核心认知框架】

- 题目本质：考查 Java 语言核心设计哲学与 C/C++ 的区别
- 关键区分点：内存安全（GC）vs 手动管理，单继承 vs 多继承
- 命题人意图：测试对 Java 安全性与简化性设计的理解

【选项原理深度拆解】

- **A.** 指针运算—错误（安全性设计）
  - 字节码铁证：JVM 指令集（如 `aload`, `astore`）操作的是引用（Reference），而非内存地址数值

```
int[] arr = new int[10];
// JVM 内部：arr 存储的是 Heap 对象的引用句柄，无法进行
   arr++ 操作
```

    JVM 保障：JVM 规范禁止直接操作内存地址，防止缓冲区溢出攻击
  - 设计哲学：移除指针运算以增强安全性（Security）和健壮性（Robustness）
- **B.** 多重继承—错误（避免菱形问题）
  - 类继承规则：Java 类仅支持单继承（`extends` 只能跟一个父类）
  - 接口弥补：通过接口（`implements` 多个）实现多重继承的效果，但避免状态冲突
  - 字节码：Class 文件结构中 `super_class` 仅有一个索引，`interfaces` 为数组
- **C.** 自动内存管理—正确（**GC** 机制）
  - 机制：JVM 垃圾收集器（Garbage Collector）自动回收不可达对象
  - 字节码：`new` 指令分配内存，无需 `free/delete`
  - 优势：防止内存泄漏（Memory Leak）和悬空指针（Dangling Pointer）
- **D.** 预处理器—错误（编译模型不同）
  - 对比：C/C++ 有 `#include`, `#define` 预处理阶段
  - **Java** 模型：编译器直接处理源文件，常量用 `final`，导入用 `import`
  - 原因：保持语言语义清晰，避免宏展开带来的调试困难

【应背诵知识点（命题人思维版）】

(a) **Java** 三大特性：简单性（无指针/预处理器）、安全性（GC/校验）、跨平台（JVM）

(b) 继承模型：类单继承，接口多实现

(c) 选项辨析速查表：

选项	**Java** 支持	错误根源	记忆口诀
指针运算	否	安全机制	”Java 无指针”
多重继承	否	类结构限制	” 类单接多”
自动内存	是	-	”GC 自动收”
预处理器	否	编译模型	” 无宏定义”

【命题趋势与实战策略】

- 考场秒杀：见” 指针/多重继承/预处理” 直接排除，选”GC/跨平台/面向对象”

2. 在 **Java** 中，哪个关键字用于引入包中的类？

a) package　　b) import　　c) include　　d) using

答案与深度解析 答案：**B. import**

【核心认知框架】

- 题目本质：考查 Java 编译单元结构及命名空间管理
- 关键区分点：`package` 定义包，`import` 引入包，C/C++ 用 `include`
- 命题人意图：测试对 Java 源文件基本语法的记忆

【选项原理深度拆解】

- **A. package** —错误（定义而非引入）
  - 功能：声明当前类所属的命名空间
  - 位置：必须位于源文件第一行（注释除外）
  - 字节码：决定 Class 文件在文件系统中的目录结构
- **B. import** —正确（引入外部类）
  - 功能：告诉编译器去哪里查找未限定的类名
  - 编译期行为：编译器将简单类名替换为全限定名，不增加运行时开销
  - 示例：`import java.util.List;`
- **C. include** —错误（**C/C++** 语法）
  - 来源：C/C++ 预处理器指令，用于包含头文件
  - **Java** 替代：Java 通过类加载机制动态加载类，无需物理包含
- **D. using** —错误（**C#/Python** 语法）
  - 来源：C# 用于引入命名空间，Python 用于导入模块
  - **Java** 对应：Java 严格使用 `import`

【应背诵知识点】

- `package` 定义包，`import` 引入类
- `java.lang` 包自动导入，无需显式 `import`

3. 下列关于变量的默认值，错误的是？

a) int 类型变量的默认值是 0　b) boolean 类型变量的默认值是 false　c) 引用类型变量的默认值是 null　d) 局部变量有默认值

答案与深度解析 答案：**D.** 局部变量有默认值

【核心认知框架】

- 题目本质：考查 JVM 内存模型中不同变量区域的初始化规则
- 关键区分点：成员变量（自动初始化）vs 局部变量（必须显式初始化）
- 命题人意图：测试对编译期检查与运行时默认值的区分

【选项原理深度拆解】

- **A. int** 默认 **0** —正确（成员变量）
  - **JVM** 规范：JVMS §2.3.2 规定类/实例变量在准备阶段初始化为 0
  - 字节码：`getfield` 读取未赋值成员变量得到 0
- **B. boolean** 默认 **false** —正确（成员变量）
  - 规范：布尔类型成员变量默认值为 `false`
  - 内存：占用 1 字节或 4 字节（取决于 JVM 实现），值初始为 0 (false)
- **C.** 引用默认 **null** —正确（成员变量）
  - 规范：所有引用类型（对象、数组、接口）默认值为 `null`
  - 含义：表示不指向任何堆对象
- **D.** 局部变量有默认值—错误（编译错误）
  - 栈帧结构：局部变量表（Local Variable Table）不会自动清零
  - 编译检查：Javac 进行数据流分析，若使用未初始化局部变量，报”variable might not have been initialized”
  - 字节码：`iload` 指令加载未初始化变量会导致验证错误

【应背诵知识点】

- 成员变量：有默认值（0/false/null）
- 局部变量：无默认值，必须初始化
- 数组元素：视为成员变量，有默认值

4. 下列运算符中，优先级最高的是？

a) +　b)

c) ++　d) =

答案与深度解析 答案：**C. ++** 【核心认知框架】：考查运算符优先级表。【深度解析】：

- 优先级：单目运算符（++）> 算术（*）> 算术（+）> 赋值（=）。

- 字节码：`iinc` (自增) 优先于 `imul`, `iadd`, `istore`。
- 陷阱：后缀 ++ 返回值后再自增，前缀 ++ 先自增再返回，但优先级均高于加减乘。

5. 关于 `break` 和 `continue` 的说法，正确的是？

a) break 只能用于 switch 语句　b) continue 只能用于循环语句　c) break 和 continue 都可以带标签　d) continue 可以用于 switch 语句

答案与深度解析 答案：**C. break 和 continue 都可以带标签** 【核心认知框架】：考查跳转语句的作用域。【深度解析】：

- **break**：可用于 switch 和循环，可带标签跳出多层。
- **continue**：仅用于循环，可带标签跳过外层循环迭代。
- 字节码：均对应 `goto` 指令，标签对应字节码偏移量。
- 错误项：break 可用于循环，continue 不可用于 switch。

6. 下列哪个选项是正确的二维数组声明？

a) `int a[3][4];`　b) `int[][] a = new int[3][];`　c) `int a[][] = new int[][4];`　d) `int[][] a = new int[][];`

答案与深度解析 答案：**B. `int[][] a = new int[3][];`** 【核心认知框架】：考查数组初始化规则。【深度解析】：

- 规则：Java 数组是对象，第一维长度必须指定，后续维可选。
- **A** 错：声明时不能指定长度（C 风格），长度在 `new` 时指定。
- **C** 错：第一维长度缺失。
- **D** 错：动态初始化必须指定至少第一维长度。
- **B** 对：不规则数组合法，第一维 3 个引用，每个引用可指向不同长度数组。

7. 关于构造方法的描述，正确的是？

a) 构造方法必须用 void 声明返回类型　b) 构造方法可以被继承　c) 构造方法可以重载　d) 构造方法可以被子类重写

答案与深度解析 答案：**C. 构造方法可以重载** 【核心认知框架】：考查构造方法语法规则。【深度解析】：

- **A** 错：构造方法不能有返回值类型（包括 void），否则变普通方法。
- **B** 错：构造方法不可继承，子类需通过 `super()` 调用。
- **D** 错：重写（Override）针对实例方法，构造方法名称随类名变化，无法重写。
- **C** 对：支持重载（Overload），参数列表不同即可。

8. 在 **Java** 中，使用哪个关键字表示当前对象的引用？

a) super　　b) this　　c) self　　d) current

答案与深度解析 答案：**B. this**【核心认知框架】：考查对象引用关键字。【深度解析】：
- **this**：指向当前实例对象引用。
- **super**：指向父类部分引用。
- **self/current**：Java 中无此关键字（Python 用 self）。
- 字节码：实例方法第一个隐藏参数即为 `this`。

9. 下列哪个访问修饰符的成员在同一个包中可见，且对子类可见（无论子类在哪个包中）？

a) public　　b) protected　　c) private　　d) 默认（无修饰符）

答案与深度解析 答案：**B. protected**【核心认知框架】：考查访问控制权限表。【深度解析】：
- **public**：所有可见。
- **protected**：同包 + 不同包子类可见（符合题意）。
- **private**：仅本类可见。
- **default**：仅同包可见，不同包子类不可见。
- 字节码：ACC_PROTECTED 标志位控制访问检查。

10. 关于静态方法，以下说法正确的是？

a) 静态方法可以直接访问非静态成员　　b) 静态方法中可以使用 this 关键字　　c) 静态方法可以被重写　　d) 静态方法可以通过类名直接调用

答案与深度解析 答案：**D.** 静态方法可以通过类名直接调用 【核心认知框架】：考查 static 语义。【深度解析】：

- **A/B** 错：静态上下文无 `this`，无法访问实例成员。
- **C** 错：静态方法可隐藏（Hide），不可重写（Override），动态绑定失效。
- **D** 对：`ClassName.method()`，编译期绑定。
- 字节码：调用指令为 `invokestatic`。

11. 下列哪个类是所有 **Java** 异常类的父类？

a) Error　　b) Exception　　c) RuntimeException　　d) Throwable

答案与深度解析 答案：**D. Throwable** 【核心认知框架】：考查异常继承树。【深度解析】：

- **Throwable**：根类，子类为 Error 和 Exception。
- **Error**：系统错误，不可捕获。
- **Exception**：程序异常，可捕获。
- 字节码：`athrow` 指令抛出 Throwable 及其子类。

12. 关于 `try-catch-finally` 结构，以下说法正确的是？

a) catch 块可以有多个，且父类异常必须放在子类异常之后　　b) finally 块可以没有　　c) try 块后面必须同时有 catch 和 finally　　d) catch 块可以单独存在，不需要 try 块

答案与深度解析 答案：**A. catch** 块可以有多个，且父类异常必须放在子类异常之后 【核心认知框架】：考查异常处理语法规则。【深度解析】：

- **A** 对：捕获顺序必须从具体到抽象，否则子类 catch 不可达。
- **B** 对（但 **A** 更核心）：finally 可选，但 try 必须配 catch 或 finally。
- **C** 错：try 后至少跟一个 catch 或 finally。
- **D** 错：catch 不能独立存在。
- 注意：题目问" 正确的是"，A 是核心规则，B 也是对的但通常 A 是考点。若单选，A 描述更具体准确关于异常匹配。* 修正：B 也是正确的陈述，但 A 是关于 catch 顺序的关键考点。通常此类题 A 为最佳。*

13. 下列哪个集合类是有序的，且允许重复元素？

a) HashSet　　b) TreeSet　　c) ArrayList　　d) HashMap

答案与深度解析 答案：**C. ArrayList** 【核心认知框架】：考查集合接口特性。【深度解析】：

- **ArrayList**：List 实现，有序（插入顺序），可重复。
- **HashSet**：Set 实现，无序，不可重复。
- **TreeSet**：Set 实现，排序，不可重复。
- **HashMap**：Map 实现，键不可重复。

14. 关于泛型，以下说法错误的是？

a) 泛型可以用于类、接口和方法　　b) 泛型可以提高类型安全性　　c) 泛型在运行时会被类型擦除　　d) 泛型可以用于基本数据类型

答案与深度解析 答案：**D.** 泛型可以用于基本数据类型 【核心认知框架】：考查泛型限制。【深度解析】：

- **D** 错：泛型类型参数必须是引用类型（如 `Integer`），不能是 `int`。
- **A/B/C** 对：泛型支持类/接口/方法，编译期检查类型，运行时擦除为 Object。
- 字节码：Signature 属性记录泛型信息，运行时不可见。

15. 在 **Java** 多线程中，`start()` 方法和 `run()` 方法的区别是？

a) start() 启动新线程并执行 run() 方法，run() 只是普通方法调用　　b) start() 和 run() 的作用相同　　c) run() 启动新线程，start() 只是普通方法　　d) 必须先调用 run() 再调用 start()

答案与深度解析 答案：**A. start()** 启动新线程并执行 **run()** 方法，**run()** 只是普通方法调用 【核心认知框架】：考查线程启动机制。【深度解析】：

- **start()**：请求 JVM 创建新线程，新线程调用 run()。
- **run()**：普通方法，在当前线程执行，无并发效果。
- 字节码：`start()` native 方法，`run()` 普通调用。

16. 关于线程同步，以下说法错误的是？

a) synchronized 关键字可以修饰方法　　b) synchronized 关键字可以修饰代码块　　c) 使用 synchronized 可以保证原子性和可见性　　d) 使用 synchronized 可以提高程序性能

答案与深度解析 答案：**D.** 使用 **synchronized** 可以提高程序性能【核心认知框架】：考查同步开销。【深度解析】：

- **D** 错：同步涉及锁竞争、上下文切换，会降低吞吐量，而非提高性能。
- **A/B/C** 对：synchronized 保证原子性、可见性、有序性。
- 字节码：`monitorenter/monitorexit` 指令。

17. 在 **Java I/O** 中，`BufferedWriter` 类的作用是？

a) 提供缓冲功能，提高字符写入效率　b) 提供字节写入功能　c) 提供对象序列化功能　d) 提供文件随机访问功能

答案与深度解析 答案：**A.** 提供缓冲功能，提高字符写入效率【核心认知框架】：考查 IO 流分类。【深度解析】：

- **BufferedWriter**：字符缓冲输出流，减少磁盘 IO 次数。
- 字节流：OutputStream 系列。
- 序列化：ObjectOutputStream。
- 随机访问：RandomAccessFile。

18. 关于序列化，`transient` 关键字的作用是？

a) 标记变量为不可序列化　b) 标记变量为临时变量，不会被垃圾回收　c) 标记变量为易失变量，保证可见性　d) 标记变量为线程局部变量

答案与深度解析 答案：**A.** 标记变量为不可序列化【核心认知框架】：考查序列化机制。【深度解析】：

- **transient**：序列化时跳过该字段，反序列化后为默认值。
- **volatile**：保证可见性（C 选项描述）。
- **ThreadLocal**：线程局部变量（D 选项描述）。

19. 在反射中，获取 **Class** 对象的三种方式不包括？

a) 类名.class　b) 对象.getClass()　c) Class.forName("完整类名")　d) new Class()

答案与深度解析 答案：**D. new Class()**【核心认知框架】：考查反射 API。【深度解析】：

- **Class** 类：构造方法私有，无法 `new`。
- 三种方式：`.class`, `getClass()`, `forName()`。
- 字节码：`ldc` 指令加载 Class 对象。

20. JDBC 中，`PreparedStatement` 与 `Statement` 相比，优势不包括？

a) 防止 SQL 注入　b) 预编译，提高执行效率　c) 支持批量操作　d) 可以执行任意复杂的 SQL 语句

答案与深度解析 答案：**D.** 可以执行任意复杂的 **SQL** 语句 【核心认知框架】：考查 JDBC 接口区别。【深度解析】:

- **D** 错：两者都能执行复杂 SQL，这不是 `PreparedStatement` 的独有优势。
- **A/B/C** 对：预编译防注入、效率高、支持批量 `addBatch`。

二、判断题深度解析（共 10 小题）

1. Java 虚拟机（**JVM**）负责将字节码转换为特定平台的机器码。

答案与深度解析 答案：正确 (✓) 【核心认知框架】：考查 JVM 核心功能。【深度解析】:

- 机制：解释器逐行解释，JIT 编译器将热点代码编译为本地机器码。
- 跨平台：字节码统一，JVM 适配各 OS。

2. 在同一个类中，可以定义两个参数列表相同但返回值类型不同的方法。

答案与深度解析 答案：错误 (×) 【核心认知框架】：考查方法重载规则。【深度解析】:

- 规则：重载仅看参数列表，返回值不同不足以区分，编译报错。
- 字节码：方法签名（Signature）不包含返回值。

3. 子类可以访问父类的 **protected** 成员。

答案与深度解析 答案：正确 (✓) 【核心认知框架】：考查访问控制。【深度解析】:

- 规则：protected 对同包和子类可见。

4. 抽象类不能实例化，但可以有构造方法。

答案与深度解析 答案：正确 (✓) 【核心认知框架】：考查抽象类特性。【深度解析】：
- 构造方法：供子类调用（super），初始化父类部分。
- 实例化：不能 `new AbstractClass()`。

5. 接口中的默认方法（**default method**）可以有方法体。

答案与深度解析 答案：正确 (✓) 【核心认知框架】：考查 JDK8 接口新特性。【深度解析】：
- **default**：允许接口提供默认实现，减少实现类代码。

6. 使用 `throws` 关键字可以在方法内部抛出异常。

答案与深度解析 答案：错误 (×) 【核心认知框架】：考查异常关键字区别。【深度解析】：
- **throws**：声明方法可能抛出的异常（方法签名处）。
- **throw**：在方法内部抛出具体的异常对象。

7. `HashSet` 中的元素按插入顺序存储。

答案与深度解析 答案：错误 (×) 【核心认知框架】：考查集合有序性。【深度解析】：
- **HashSet**：无序（基于 HashMap）。
- **LinkedHashSet**：按插入顺序。

8. 泛型类型参数在运行时会被擦除，因此无法在运行时获取泛型的具体类型。

答案与深度解析 答案：正确 (✓) 【核心认知框架】：考查类型擦除。【深度解析】：
- 擦除：`List<String>` 运行时变为 `List`。
- 反射：无法通过 `getClass()` 获取泛型参数。

9. 线程调用 `wait()` 方法后，会释放持有的对象锁。

答案与深度解析 答案：正确 (✓) 【核心认知框架】：考查线程通信。【深度解析】：
- **wait()**：进入等待集，释放锁，允许其他线程进入同步块。
- **sleep()**：不释放锁。

10. `File` 类的 `exists()` 方法可以判断文件或目录是否存在。

答案与深度解析 答案：正确 **(✓)** 【核心认知框架】：考查 File API。【深度解析】：

- **exists()**：返回 boolean，检查路径是否存在。

三、填空题深度解析（共 5 小题）

1. **Java** 的基本数据类型中，字符类型是 ____，布尔类型是 ____。

> 答案与深度解析 答案：**char**；**boolean**【核心认知框架】：考查基本类型关键字。【深度解析】：
> - **char**：16 位 Unicode 字符。
> - **boolean**：真/假逻辑值。
> - 注意：Java 中类型关键字均小写。

2. 在面向对象中，____ 是指一个类具有多种形态，包括方法重载和 ____。

> 答案与深度解析 答案：多态性；方法重写（或覆盖）【核心认知框架】：考查多态定义。【深度解析】：
> - 多态：编译时多态（重载），运行时多态（重写）。
> - 核心：父类引用指向子类对象。

3. 使用 ____ 关键字可以调用父类的构造方法，使用 ____ 关键字可以调用本类的其他构造方法。

> 答案与深度解析 答案：**super**；**this**【核心认知框架】：考查构造器调用。【深度解析】：
> - **super()**：必须第一行，调用父类构造。
> - **this()**：必须第一行，调用本类重载构造。
> - 互斥：两者不能同时出现在一个构造器中。

4. 线程的五种状态包括新建、____、运行、____ 和死亡。

> 答案与深度解析 答案：就绪；阻塞【核心认知框架】：考查线程生命周期。【深度解析】：
> - 状态：New, Runnable, Running, Blocked/Waiting, Terminated。
> - 转换：start() -> 就绪，调度 -> 运行，wait/sleep -> 阻塞。

5. 集合框架中，`List` 接口的常用实现类有 ____ 和 ____。

答案与深度解析 答案：**ArrayList**；**LinkedList** 【核心认知框架】：考查集合实现类。【深度解析】：

- **ArrayList**：数组，查询快。
- **LinkedList**：链表，增删快。
- **Vector**：线程安全，较少用。

四、程序运行结果题深度解析（共 5 小题）

1. 写出下列程序的输出结果：

```
public class Test1 {
public static void main(String[] args) {
int a = 5, b = 7;
while (a < b) {
a++;
b--;
}
System.out.println("a=" + a + ", b=" + b);
}
}
```

答案与深度解析 答案：`a=6, b=6` 【核心认知框架】：考查循环终止条件。【执行流程深度拆解】:

- 初始：a=5, b=7。
- 第 **1** 轮：5<7 真，a=6, b=6。
- 第 **2** 轮：6<6 假，循环终止。
- 输出：a=6, b=6。

2. 写出下列程序的输出结果：

```
class Parent {
public void method() { System.out.print("Parent "); }
}
class Child extends Parent {
public void method() { System.out.print("Child "); }
}
public class Test2 {
public static void main(String[] args) {
Parent p = new Child();
p.method();
}
}
```

答案与深度解析 答案：`Child` 【核心认知框架】：考查多态动态绑定。【执行流程深度拆解】：

- 引用：p 是 Parent 类型。
- 对象：实际是 Child 实例。
- 绑定：实例方法动态绑定，调用子类重写版本。
- 字节码：`invokevirtual` 指令查找实际对象类型的方法表。

3. 写出下列程序的输出结果：

```
public class Test3 {
public static void main(String[] args) {
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");
System.out.println(s1 == s2);
System.out.println(s1 == s3);
}
}
```

答案与深度解析 答案：`true`
`false` 【核心认知框架】：考查字符串池与堆内存。【执行流程深度拆解】：

- **s1, s2**：字面量，指向字符串池中同一对象，== 为 true。
- **s3**：`new` 在堆中新建对象，地址不同，== 为 false。
- 内容：`equals()` 均为 true。

4. 写出下列程序的输出结果：

```
public class Test4 {
public static void main(String[] args) {
int[] arr = {1, 2, 3, 4, 5};
int sum = 0;
for (int i = 0; i < arr.length; i += 2) {
sum += arr[i];
}
System.out.println(sum);
}
}
```

答案与深度解析 答案：9 【核心认知框架】：考查数组遍历与步长。【执行流程深度拆解】：

- 索引：0, 2, 4 (i+=2)。
- 值：arr[0]=1, arr[2]=3, arr[4]=5。
- 求和：1+3+5 = 9。

5. 写出下列程序的输出结果：

```
class MyClass {
int x;
static int y;
MyClass(int x) {
this.x = x;
y++;
}
}
public class Test5 {
public static void main(String[] args) {
MyClass obj1 = new MyClass(5);
MyClass obj2 = new MyClass(10);
System.out.println(obj1.x + " " + obj2.x);
System.out.println(MyClass.y);
}
}
```

答案与深度解析 答案：5 10

2 【核心认知框架】：考查实例变量与静态变量区别。【执行流程深度拆解】：

- **x**：实例变量，obj1.x=5, obj2.x=10，互不影响。
- **y**：静态变量，类共享，构造两次，y 自增两次，结果为 2。

五、编程题深度解析与设计模式分析（共 2 小题）

1. 设计一个" 几何图形" 类及其子类

答案与深度解析 核心设计思路：抽象类 + 多态数组

【核心认知框架】

- 题目本质：考查抽象类定义、方法重写及多态调用
- 关键区分点：抽象方法无实现，子类必须实现；父类引用指向子类对象
- 命题人意图：测试面向对象核心设计能力（抽象与多态）

【关键技术点深度拆解】

- 抽象类 Shape：
  - 定义契约 area() 和 perimeter()
  - 不能实例化，强制子类实现行为
- 子类实现：
  - Circle：面积 $= \pi \times r^2$
  - Rectangle：面积 $= w \times h$
  - 必须提供构造方法初始化属性
- 多态数组：
  - Shape[] shapes 存储不同子类对象
  - 循环调用 shapes[i].area() 动态绑定

【参考代码核心片段】

```
abstract class Shape {
    abstract double area();
    abstract double perimeter();
}
class Circle extends Shape {
    double r;
    Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }
    double perimeter() { return 2 * Math.PI * r; }
}
// 测试类中
```

```java
Shape[] shapes = new Shape[4];
shapes[0] = new Circle(1);
// 遍历调用
for(Shape s : shapes) { System.out.println(s.area()); }
```

2. 实现一个"学生成绩"管理系统

答案与深度解析 核心设计思路：**POJO** + 集合框架 + 算法

【核心认知框架】

- 题目本质：考查类设计、集合操作及排序算法
- 关键区分点：ArrayList 增删查，Collections.sort 定制排序
- 命题人意图：测试综合业务逻辑实现能力

【关键技术点深度拆解】

- **Student** 类：
  - 私有属性 + Getter/Setter
  - 重写 toString() 方便打印
- **GradeManager** 类：
  - 内部维护 ArrayList<Student>
  - removeStudent：遍历查找 id 移除
  - getTopStudents：排序后取前 n 个
- 排序逻辑：
  - 使用 Comparator 或实现 Comparable
  - 先按成绩降序，再按学号升序

【参考代码核心片段】

```java
class Student {
    String id; double score;
    // 构造/get/set/toString
}
class GradeManager {
    List<Student> list = new ArrayList<>();
    void addStudent(Student s) { list.add(s); }
    List<Student> getTopStudents(int n) {
        list.sort((a, b) -> Double.compare(b.score, a.score)
            );
        return list.subList(0, Math.min(n, list.size()));
```

```
    }
}
```

——讲义结束，祝学习顺利——